



PCF / TAS Certificate Rotation Guide

A reference for what the various certificates on a TAS foundation are for, how each one is rotated, and how long the rotation takes. The time estimates use the **CHD Prod** foundation as the worked example — its measured instance-group sizes (e.g. `router` = 6, `tcp_router` = 6, `uaa` = 3, `grafana` = 1) feed the per-VM rotation model, and the foundation-wide (CA) figures are scaled to its VM count. For another foundation, substitute that foundation's per-instance-group VM counts; the calculation is identical.

The cost model (how every number below is derived)

A certificate is “rotated” by changing it in Ops Manager and running **Apply Changes**. The cost of an Apply Changes is dominated by how many VMs BOSH has to recreate/restart, because BOSH only touches the instance groups whose certificate content actually changed.

```
time for ONE Apply Changes over an instance group
  = 20 min (fixed overhead: staging, compile, manifest, migrations)
  + (serial recreate waves) × (4–6 min per VM)

serial recreate waves = canaries + ceil((VMs - canaries) / max_in_flight)
```

- **4 min/VM** is the optimistic (low) figure, **6 min/VM** the conservative (high).
- `canaries` and `max_in_flight` come from each instance group's BOSH manifest `update` block. For safety-critical groups (routers, uaa, databases) these are usually `max_in_flight: 1`, so the group rolls **one VM at a time** — i.e. waves = number of VMs.
- Groups marked `serial: false` can roll in parallel with their peers, so the real wall-clock can come in **under** these (deliberately conservative, serial) numbers.

On CHD Prod: the **Diego cells** are the **only** instance group with `max_in_flight: 5` (they roll 5 VMs at a time); **every other instance group rolls serially** (`max_in_flight: 1`, one VM at a time). So a leaf rotation of the front-door groups (`router`, `uaa`, `grafana`) is one wave per VM, and the cell fleet is the *only* source of parallelism anywhere on the foundation. The cells aren't touched by a leaf rotation, but a foundation-wide / CA rotation recreates *every* VM — so the cells' 5-at-a-time roll is what keeps a whole-foundation apply from being a fully serial slog (CHD's foundation-wide apply ≈ 21h 44m – 32h 26m for 514 VMs, well under the ~34h–52h a fully serial roll would take).

The example minute figures below use the CHD foundation’s measured instance-group sizes (`router` = 6, `tcp_router` = 6, `uaa` = 3, `grafana` = 1, all rolling serially). On a different foundation, substitute that foundation’s per-IG VM counts — the *shape* of the calculation is identical.

Two axes that decide everything: leaf vs CA, configurable vs non-configurable

Leaf certificate vs CA certificate

	Leaf certificate	CA certificate
What it is	A server/client TLS cert presented on a connection (e.g. the gorouter’s HTTPS cert)	A signing authority whose signature other parties trust
Rotation	1 Apply Changes — regenerate the cert, recreate only the instance group(s) that use it	3 Apply Changes (the add → re-sign → remove dance — see below)
Blast radius	Only the consuming instance group(s)	Every VM that <i>trusts</i> the CA (often the whole foundation)
Cost driver	The VMs of the consuming instance group	The number of foundation-wide applies × every VM

Configurable vs non-configurable

- **Non-configurable (auto-generated):** TAS/Ops Manager generates and signs these itself (from an internal CA). You don’t supply anything — when the signing CA rotates, or the leaf expires, the platform regenerates them automatically. Most internal mesh certs (`diego`, `cloud_controller`, `nats`, `loggregator`, `credhub`, ...) are non-configurable. **Rotation is just an Apply Changes; no external action.**
- **Configurable (operator-supplied):** You provide the certificate yourself — typically a public cert bought from a CA such as **Digicert** (front-door TLS), or your org’s own CA (trust store). The platform cannot regenerate these for you, so rotation is a **two-part job**: (1) obtain the new cert from your CA *out-of-band*, then (2) upload it and Apply Changes. The “obtain from Digicert” step is not included in any Apply-Changes time estimate — it’s lead time you must plan for separately.

In a report, a configurable cert is flagged “**requires Digicert.**”

Certificate lifetimes (how long until it expires)

The **validity period** (how long a cert is good for) is set when the cert is issued and is independent of how long it takes to *rotate*. Use these standard durations — and **do not exceed them**:

Cert kind	Standard validity (do not exceed)	Notes
CA certificates — root CA, services TLS CA, BOSH DNS CA, NATS CA, Diego instance-identity CA, intermediates	4 years	A CA should be issued for at most 4 years . Intermediates chain under their root, so keep them at or below the root's remaining life.
Leaf certificates — internal auto-generated leaves (diego, cloud_controller, nats, loggregator, credhub, ...)	2 years	A leaf should be issued for at most 2 years .
Operator-supplied / public leaf — PoE front-door, UAA SP key, grafana (the Digicert ones)	≤ ~13 months (398 days)	Public CAs cap TLS-cert lifetime per CA/Browser-Forum rules, so these are renewed ~annually — already well under the 2-year leaf ceiling, and almost always the soonest to expire.

Two practical consequences:

- **The configurable (Digicert) leaf certs expire first and most often.** They are your recurring, roughly-annual rotation work, and each carries out-of-band lead time (you must buy the renewal before you can Apply Changes).
- **CAs expire rarely but are the expensive rotations.** At a 4-year ceiling you get years of warning, but the rotation itself is a multi-day, foundation-wide, 3-phase event — so it must be planned well ahead of the expiry date.

Durations are configurable (CredHub / Ops Manager settings); the values above are the recommended ceilings — **CAs ≤ 4 years, leaves ≤ 2 years**. The authoritative expiry for any cert is what the Ops Manager certificates API reports.

Certificate reference

Networking PoE (Point of Entry) TLS certificate

- **Property:** `.properties.networking_poe_ssl_certs[...].certificate`

- **Used by:** the **gorouter** (`router`) — this is the front-door HTTPS cert that TLS-terminates all app and system traffic entering the foundation. The conservative scope also includes `tcp_router` and `ha_proxy` where present.
- **Configurable: Yes** — operator-supplied (your public wildcard cert, e.g. `*.sys.<domain>` / `*.apps.<domain>` from Digicert).
- **Rotation: Leaf — 1 Apply Changes** over the router instance group(s).
- **Time (CHD example):** `router` 6 + `tcp_router` 6 = 12 VMs → $20 + 12 \times (4-6) = \approx 1\text{h } 08\text{m} - 1\text{h } 32\text{m}$.
- **Note:** because this terminates *all* ingress, the rotation recreates the routers — plan it as a rolling event (no full outage, but each router drains in turn).

UAA SAML Service Provider key / credentials

- **Property:** `.uaa.service_provider_key_credentials`
- **Used by:** the `uaa` instance group (it is colocated on `control` in a small-footprint install). This is the key UAA uses as a SAML service provider.
- **Configurable: Yes** (operator-supplied).
- **Rotation: Leaf — 1 Apply Changes** over the `uaa` group.
- **Time (CHD example):** `uaa` = 3 VMs → $20 + 3 \times (4-6) = \approx 32\text{m} - 38\text{m}$.

Routing custom CA certificates

- **Property:** `.properties.routing_custom_ca_certificates`
- **Used by:** the **gorouter** (`router`) — the CA list the router trusts when talking to custom/backends. Despite the name it behaves as a router-scoped trust list, not a foundation-wide trust anchor.
- **Configurable: Yes.**
- **Rotation: Leaf-style — 1 Apply Changes** over the `router` group.
- **Time (CHD example):** `router` = 6 VMs → $20 + 6 \times (4-6) = \approx 44\text{m} - 56\text{m}$.

Grafana route TLS certificate

- **Property:** `.properties.grafana_route.manual.ssl_certificates`
- **Used by:** the `grafana` instance group in the **Healthwatch** tile (the Grafana dashboard's HTTPS cert).
- **Configurable: Yes.**
- **Rotation: Leaf — 1 Apply Changes** over the `grafana` group.
- **Time (CHD example):** `grafana` = 1 VM → $20 + 1 \times (4-6) = \approx 24\text{m} - 26\text{m}$.

Internal TAS leaf certificates (the large majority)

- **Properties:** `.properties.<component>_*` — e.g. `bbs_*`, `cloud_controller_*`, `diego_*`, `nats_*`, `loggregator_*`, `credhub_tls`, `log_cache_*`, etc. (often **100+** on a full foundation).
- **Used by:** their owning instance group(s) within the `cf` deployment.

- **Configurable: No** — auto-generated and signed by internal TAS CAs.
- **Rotation: Leaf — 1 Apply Changes**, scoped to the consuming instance group(s). Because there are many and they're spread across the cf tile, rotating them together typically recreates much of the cf deployment.
- **Time:** scales with the affected instance groups' VM counts (same formula).

Services TLS CA

- **Property / name:** `/services/tls_ca` (the "Services TLS CA Procedure").
- **Used by:** the **on-demand service instances** (RabbitMQ, Redis, MySQL, Spring Cloud Services, the `service-instance_*` deployments, ...) and internal service TLS. It signs the leaf certs those services present. **The `cf` tile itself does not use any leaf certs signed by this CA.**
- **Configurable:** It is a CA (operator-relevant) and is flagged requires-Digicert when supplied externally.
- **Rotation: CA — 3 Apply Changes**, but with a special phase-2 scope:
 1. **Phase 1 — propagate the new CA:** foundation-wide (every tile) so all components trust both old and new CA.
 2. **Phase 2 — regenerate its leaf certs: every tile EXCEPT `cf`** (cf uses none of this CA's leaves).
 3. **Phase 3 — delete the old CA:** foundation-wide (every tile).
- **Time:** $2 \times (\text{foundation-wide apply}) + 1 \times (\text{all tiles except cf})$. On a large foundation this is the single most expensive rotation after a root CA. (On OMA: $\approx 61\text{h} - 91\text{h}$ for the full 3-phase campaign over ~558 VMs.)

Diego instance-identity CA

- **Name:** `/cf/diego-instance-identity-root-ca` and its intermediate `/p-bosh/.../diego-instance-identity-intermediate-ca`.
- **Used by:** the **Diego cells** — this CA issues the short-lived **per-app-container identity certificates** (app instance identity and container-to-container mTLS). The leaf identity certs are issued *dynamically* by each cell and are very short-lived, so they regenerate automatically as containers cycle — you never rotate those by hand.
- **Configurable: No** (internally generated).
- **Rotation: CA — 3 Apply Changes, scoped to the `cf` deployment only** (the canonical *deployment-scoped* CA). Rotating the CA re-establishes the chain the cells use to mint container certs; the dynamic leaf certs follow automatically.
- **Time:** $3 \times (\text{one apply over the cf deployment})$.
- **Standard validity:** it is a CA — issue for **at most 4 years**; keep the intermediate at or below the root.

NATS client CA

- **Property:** `.properties.nats_client_ca` (with leaves `nats_client_cert`, `nats_tls_*`).
- **Used by:** the **NATS message bus** mTLS — components such as the gorouter and route-emitter authenticate to NATS with certs signed by this CA. Foundation- internal (component-to-component), not operator-facing.
- **Configurable:** **No** (internally generated).
- **Rotation:** **CA — 3 Apply Changes, foundation-wide** (a foundation trust anchor for the control-plane message bus).
- **Time:** `3 × (foundation-wide apply)`.
- **Standard validity:** it is a CA — issue for **at most 4 years**.

Root CA / Ops Manager Root CA

- **Name:** `.properties.root_ca`, `opsman-root-ca:...`, and similar foundation trust anchors.
- **Used by:** **every VM** — it is the trust anchor underneath the platform's internal PKI. Effectively everything chains back to it.
- **Configurable:** **Yes** (you *can* supply your own root CA), but it is normally **internally generated**, so a configurable-but-internal root CA is **not** flagged requires-Digicert.
- **Rotation:** **CA — 3 Apply Changes, all foundation-wide** (see the 3-phase explanation below). This is the most expensive rotation on the foundation: each of the 3 applies recreates *every* VM.
- **Time:** `3 × (foundation-wide apply)`. (On OMA, one foundation-wide apply $\approx$ **24h – 36h**, so a root CA rotation $\approx$ **72h – 108h** of Apply-Changes compute, excluding change-window gaps.)

BOSH trusted-certificates (trust store)

- **Property:** `.security_configuration.trusted_certificates[...]`
- **Used by:** **every VM** — Ops Manager distributes these CAs into every BOSH-deployed VM's OS trust store (and the director). Typically your org's CA or an external CA you want components to trust for outbound TLS.
- **Configurable:** **Yes** — externally supplied (requires-Digicert).
- **Rotation:** **Single foundation-wide Apply Changes** — *not* the 3-phase dance. The old CA is swapped for the new one in one config change, and one Apply Changes propagates it to every VM's trust store. It signs only external-service leaves, so there is no platform leaf-regen phase and no separate removal apply.
- **Time:** `1 × (foundation-wide apply)`.

Ops Manager UI SSL certificate (outside the Apply-Changes model)

- **What:** the TLS cert for the **Ops Manager web interface itself** (the operator console), not a cert on the deployed foundation.
- **Configurable:** **Yes** — you can upload your own (or use the self-signed default).
- **Rotation:** done in **Ops Manager** → **Settings** → **SSL Certificate**, **not** via a foundation Apply Changes. It does **not** recreate any deployment VMs, so it carries none of the cost in this guide —

but it's easy to forget precisely because it never shows up in the deployed-products / `cert-rotation` report.

- **Time:** effectively immediate (a UI config change + Ops Manager reload).

Per-service-tile certificates

- **What:** each marketplace **service tile** (RabbitMQ, Redis, Tanzu SQL / MySQL, Spring Cloud Services, Harbor, Healthwatch, ...) ships its **own internal CA and leaf certs** for its component VMs — distinct from the Services TLS CA, which covers the on-demand service *instances*.
- **Configurable:** mostly **No** (tile-managed); some tiles expose an operator-supplied front-end cert (e.g. Harbor registry TLS, RabbitMQ management TLS).
- **Rotation: deployment-scoped** — a leaf rotates with 1 Apply Changes over that tile's instance group(s); the tile's own CA rotates 3-phase on that tile's deployment. Cost scales with that tile's VM count.

BOSH DNS certificates — see the dedicated section below.

Why a CA rotation takes 3 Apply Changes

A CA can't be swapped in a single step without breaking live TLS. If you replaced the old CA with the new one at once, there would be a window where a component presents a certificate signed by one CA to a peer that only trusts the other — the TLS handshake fails and you get an outage. The 3-phase procedure exists to keep **both** CAs trusted throughout the transition so no live certificate is ever untrusted:

1. **Apply 1 — ADD the new CA (generate).** Generate the new CA and add it to the trust store *alongside* the old one. Now every component trusts **both** old and new. Nothing is signed by the new CA yet, so nothing changes on the wire — this apply just distributes trust.
2. **Apply 2 — ACTIVATE the new CA and regenerate the leaf certificates.** Mark the new CA as the active signer and regenerate the leaf certs so they are signed by the **new** CA. Components now *present* new-CA-signed certs. Because everyone still trusts both CAs (from Apply 1), every handshake still succeeds — zero downtime.
3. **Apply 3 — REMOVE the old CA.** Now that no live cert is signed by the old CA, remove it from the trust store. Only the new CA remains trusted. The rotation is complete.

In short: you can't simultaneously (a) trust only the new CA and (b) still be presenting old-CA certs. The middle apply is what lets you cross that gap without an outage — add-new-trust → switch-leaves → drop-old-trust. Each phase is a full Apply Changes (BOSH must redeploy the affected VMs to pick up the trust-store and cert changes), which is why a CA rotation costs **3×** a leaf rotation's footprint.

How expensive it gets depends on the CA's reach. Each of the 3 applies recreates the VMs in that phase's scope. For a **root / foundation-wide CA** (root CA, BOSH DNS CA, NATS CA) all three phases are foundation-wide — each apply recreates *every* VM — so it is the **most expensive rotation on the platform**:

`3 × one foundation-wide apply`, on the order of **72h – 108h** of Apply-Changes compute on a large (~558-VM) foundation. That's compute only; real calendar time is longer because of change-window approvals between phases, so a root CA rotation is a **multi-day, multi-window event you must plan well ahead of expiry**. A **deployment-scoped CA** (e.g. the Diego instance-identity CA) only recreates its owning deployment, and the **services TLS CA** skips the cf tile on phase 2 — see those entries in the reference.

Deferring the removal (shortens the immediate window): Apply 3 (remove old CA) is the only phase that can safely wait — an expired-but-still-trusted CA in the store is harmless. Running it in a **later maintenance window** takes one foundation-wide apply *out of the up-front rotation*, so you schedule less in one sitting. For a root CA the immediate work drops from `3 × foundation-wide` (~72h – 108h) to `2 ×` (~48h – 72h), with the remaining **~24h – 36h deferred** to the follow-up window. (It doesn't reduce the *total* compute — still three applies — but it shrinks the change window you have to fit into a single maintenance slot.)

Even better — fold it into an apply you're already doing. The old-CA removal is just a config change that BOSH picks up on the *next* Apply Changes. So if you have a foundation-wide Apply Changes coming up anyway — a stemcell update, a tile upgrade, or any other change that recreates every VM — let the removal ride along with it. The old CA drops out of the trust store as part of that deploy, so it costs **no extra apply at all**: the deferred ~24h – 36h disappears into work you were going to run regardless.

How DNS certificates are rotated

BOSH DNS gives every VM a local DNS resolver, secured by its own small PKI:

- `/opsmgr/bosh_dns/tls_ca` — the CA for BOSH DNS mutual TLS.
- **Leaf certs:** `bosh_dns_health_client_tls`, `bosh_dns_health_server_tls`, `dns_api_client_tls`, `dns_api_server_tls` — the client/server certs the bosh-dns-health and dns-api jobs present to each other.

These are non-configurable (auto-generated) and, crucially, **bosh-dns runs on every VM in the foundation** (it's colocated on every instance group, not a tile of its own). So:

- **The DNS leaf certs** rotate like any other non-configurable leaf — 1 Apply Changes — but because bosh-dns is everywhere, their consuming “instance group” is effectively **the whole foundation**. There's no narrow IG to scope to.
- **The DNS CA** (`bosh_dns/tls_ca`) rotates as a **foundation-wide 3-phase CA**, exactly like a root CA: add new (all VMs) → re-sign the bosh-dns health/api leaves (all VMs) → remove old (all VMs).

Does it fit the generic bucket? Mostly yes — the *mechanism* is the standard CA / leaf rotation. What makes DNS certs different in practice is **scope**: because bosh-dns is colocated on every VM, even the “leaf” rotation touches the entire foundation, so a DNS leaf rotation costs about the same as a foundation-wide apply rather than a small per-IG apply. Treat a DNS **CA** rotation like a root CA (3 × foundation-wide), and a DNS **leaf** rotation like one foundation-wide apply.

Recommended: automatic rotation via Director Config

The simplest way to keep the BOSH DNS CA and its leaf certs current is to let the platform rotate them for you. In **Ops Manager** → **BOSH Director tile** → **Director Config**, enable “**Enable automatic rotation of the BOSH DNS CA certificate.**”

With this turned on, the BOSH DNS CA/leaf certificates are rotated automatically, and the rotated certificates are **deployed to VMs during an Apply Changes when new stemcells are also being deployed to those VMs**. In other words it piggybacks on the stemcell/VM recreation you’re already doing — there’s no separate, dedicated DNS-cert rotation campaign to run. **Refer to the product documentation for details.**

This is the proactive, low-effort path and should be the default. The manual procedure below is only for rotating out-of-band, and the `--skip-drain` recovery further down is only for the reactive case where the CA has already expired.

Manual rotation (out-of-band): CredHub `maestro` transitional signing

If you need to rotate the DNS CA manually, the 3-phase “add new → re-sign → remove old” dance is carried out with the CredHub `maestro` CLI plus Apply Changes. The transitional-signing feature is how CredHub keeps **both** the old and new CA trusted during the cross-over (the same zero-downtime principle as any CA rotation):

Phase 1 — regenerate the CA and transition, then re-sign the leaves:

```
maestro regenerate ca --name "/opsmgr/bosh_dns/tls_ca"
maestro update-transitional signing --name "/opsmgr/bosh_dns/tls_ca"
maestro regenerate leaf --signed-by "/opsmgr/bosh_dns/tls_ca"
```

Phase 2 — deploy: run **Apply Changes** in Ops Manager (enable “*Upgrade all service instances*” so the on-demand service tiles pick up the new DNS certs too).

Phase 3 — clean up: drop the transitional (old) CA, then Apply Changes again:

```
maestro update-transitional remove --name "/opsmgr/bosh_dns/tls_ca"
```

(again with service-instance upgrades enabled).

Special case: the DNS CA has **ALREADY** expired

This is the gotcha that makes an *expired* DNS CA worse than a normal rotation. BOSH VM **drain scripts depend on a working BOSH DNS** — but if the DNS CA has already expired, DNS is broken, so the drains hang and the Apply Changes **deployment fails**. To get past it, redeploy the stuck deployment skipping drain:

```
bosh -d FAILED_DEPLOYMENT manifest > deployment.yaml  
bosh -d FAILED_DEPLOYMENT deploy --skip-drain deployment.yaml
```

Repeat per failing deployment until the new certs are on every instance, then continue with Phase 3.

Implication: rotate the BOSH DNS CA *before* it expires — a proactive rotation is a clean 3-phase Apply-Changes job; a reactive one (after expiry) adds per-deployment `--skip-drain` recovery on top.

Source: Broadcom KB 424654 — *How to rotate expired BOSH DNS CA certificates* (knowledge.broadcom.com/external/article/424654).

Quick-reference: individual rotation cost

Each certificate rotated on its own. The **Rotation time** is the total Apply-Changes compute for the whole rotation (all phases). Leaf rows use the CHD instance-group sizes; CA rows use a ~558-VM (OMA-scale) foundation — substitute your foundation's VM counts.

Certificate	Used by	Configurable	Validity	Applies	Rotation time
Networking PoE TLS	gorouter (router, tcp_router)	Yes (Digicert)	≤ ~13 mo	1	~1h 08m – 1h 32m (12 VMs)
UAA SAML SP key	uaa (control in SF)	Yes (Digicert)	≤ ~13 mo	1	~32m – 38m (3 VMs)
Routing custom CA	gorouter (router)	Yes (Digicert)	≤ ~13 mo	1	~44m – 56m (6 VMs)
Grafana route TLS	grafana (Healthwatch)	Yes (Digicert)	≤ ~13 mo	1	~24m – 26m (1 VM)
Internal TAS leaves	their owning IGs	No (auto)	2 yr max	1	scales with IG size
Per-service-tile certs	each service tile's VMs	mostly No	2 yr leaf / 4 yr CA	1–3	scales with tile size
Diego instance-identity CA	Diego cells (cf)	No	4 yr max	3	3× the cf deployment
NATS client CA	every VM (NATS bus)	No	4 yr max	3	~72h – 108h (whole foundation)
Services TLS CA	service instances	CA (Digicert)	4 yr max	3	~61h – 91h (whole foundation)
Root CA / opsman root CA	every VM	Yes (internal)	4 yr max	3	~72h – 108h (whole foundation)
BOSH DNS CA	every VM (bosh-dns)	No	4 yr max	3	~72h – 108h (whole foundation)
Trusted-store CA	every VM (trust store)	Yes (Digicert)	your org CA	1	~24h – 36h (whole foundation)
Ops Manager UI SSL	Ops Manager console	Yes	≤ ~13 mo	0	immediate (OM Settings, no Apply Changes)

The CA-row times come from the rotation model: one foundation-wide Apply Changes ≈ **24h – 36h** on a ~558-VM foundation, so a Root/DNS CA (3 phases) ≈ 72h–108h, the Services TLS CA (2 phases all-tiles + 1 phase

all-but-cf) $\approx$ 61h–91h, and the trusted-store CA (single swap apply) $\approx$ 24h–36h. Leaf-row times are one apply over just the listed instance group(s).

Key takeaways

- **Leaf rotation = 1 apply, scoped to the consuming instance group.** Cheap and fast — minutes to a couple of hours — *if* the cert is used by a small group (router, uaa, grafana). It only gets expensive when the consuming group is large or when the cert is everywhere (DNS).
- **CA rotation = 3 applies** because you must add-new-trust $\rightarrow$ re-sign-leaves $\rightarrow$ drop-old-trust to avoid a TLS outage. The last apply can be deferred.
- **Foundation-wide CAs (root CA, BOSH DNS CA) are the expensive ones** — every phase recreates every VM.
- **The services TLS CA is the special case:** 2 foundation-wide phases + 1 phase over every tile *except* cf.
- **The BOSH trusted-store CA is the cheap CA:** a single foundation-wide swap apply.
- **Configurable certs add out-of-band lead time** — you must obtain the new cert from your external CA (Digicert) before any Apply Changes; that time is *not* in the compute estimate.
- **All estimates are Apply-Changes compute time only** — they exclude the change-window / approval gaps between phases, which often dominate the real calendar time for CA rotations.